

Sentiment Analysis Assignment

PART 1 – THEORY

1. What is Sentiment Analysis?

Sentiment Analysis is a Natural Language Processing (NLP) technique used to determine whether textual data expresses a positive, negative, or neutral opinion or emotion.

2. Where can we get reviews?

Reviews can be collected from:

- Amazon & Flipkart
- Google Reviews, Yelp, TripAdvisor
- Social media (Twitter/X, Facebook, Instagram comments)
- Movie & book sites (IMDb, Goodreads)
- Customer surveys and feedback forms

3. Importance of Sentiment Analysis

- Business insights for improving products and services
- Brand reputation monitoring
- Market research
- Political opinion analysis
- Automating customer support feedback

4. Steps in Sentiment Analysis

- Data collection
- Text preprocessing

- Feature extraction (Bag-of-Words / TF-IDF)
- Model training
- Prediction
- Model evaluation

5. How do we evaluate a model?

- Accuracy
- Confusion Matrix
- Precision, Recall, F1-score

PART 2 – LEXICON SENTIMENT ANALYSIS

Sentence 1: Positive

Sentence 2: Negative

Sentence 3: Positive

Sentence 4: Positive

PART 3 – PYTHON CODE

```
import nltk

from sklearn.feature_extraction.text import CountVectorizer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
nltk.download('stopwords')
```

```
data = [  
    ("I love this product", "positive"),  
    ("This is terrible", "negative"),  
    ("I am not sure about this", "neutral"),  
    ("Amazing experience", "positive"),  
    ("Worst service ever", "negative"),  
    ("It is okay", "neutral"),  
]
```

```
X, y = zip(*data)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
vectorizer = CountVectorizer(stop_words='english')
```

```
X_train_counts = vectorizer.fit_transform(X_train)
```

```
X_test_counts = vectorizer.transform(X_test)
```

```
classifier = MultinomialNB()
```

```
classifier.fit(X_train_counts, y_train)
```

```
predictions = classifier.predict(X_test_counts)
```

```
accuracy = accuracy_score(y_test, predictions)
```

```
conf_matrix = confusion_matrix(y_test, predictions)
class_report = classification_report(y_test, predictions)

print("Accuracy:", accuracy)
print("Confusion Matrix:", conf_matrix)
print("Classification Report:", class_report)
```